

IC4302 - Tarea Corta 02

Curso: Bases de Datos II (IC4302)

Semestre: Segundo Semestre 2025

Institución: Tecnológico de Costa Rica – Escuela de Ingeniería en Computación

Instrucciones de Ejecución

► Desplegar información

1.1 Requisitos Previos

- Cuenta en Docker Hub: Es un sitio web donde puedes guardar y compartir imágenes de programas listos para usar. Es como una "nube" para aplicaciones.
- Docker y Docker Compose: Docker es una herramienta que permite ejecutar programas en "contenedores", que son como cajas que traen todo lo necesario para que el programa funcione igual en cualquier computadora. Docker Compose ayuda a iniciar varios de estos programas juntos fácilmente.
- Kubernetes (Minikube o Docker Desktop): Kubernetes es una plataforma que ayuda a administrar y ejecutar muchos contenedores a la vez, ideal para proyectos grandes. Minikube y Docker Desktop son formas sencillas de usar Kubernetes en tu propia computadora.
- Helm Charts instalados: Helm es una herramienta que facilita la instalación y actualización de aplicaciones en Kubernetes, usando "charts" que son como recetas pre-hechas.
- Git: Es una herramienta para guardar y controlar los cambios en el código de un proyecto, permitiendo trabajar en equipo y mantener un historial de versiones.
- Lens: Es un programa con interfaz gráfica que permite ver y administrar fácilmente los recursos y servicios que se están ejecutando en Kubernetes.

1.2 Instalación de Componentes

1. Descargue el repositorio del proyecto en su computadora

```
git clone <URL_REPO>
```

Después ingrese a la carpeta del repositorio por medio de la terminal bash:

```
cd 2025-02-IC4302
```

2. Construya la imagenes de docker

Para poder realizar la construcción de las imágenes Docker, debe ingresar a la carpeta **docker** desde una terminal Bash y ejecutar el siguiente comando:

```
./build.sh usuario
```

NOTA: Sustituya la palabra usuario con su usuario de Docker Hub

3. Configure el registro de las imágenes para el chart

En su proyecto, ingrese a la carpeta de charts --> app --> templates --> values.yaml y registre el nombre de usuario en docker hub que desea utilizar en el campo **docker_registry**

```
dataseeder:  
  image: chrisjimenez/dataseeder # docker registry replace with your own username
```

3.2 Configure la carga y uso de las bases de datos

En caso de desear la ejecución de solamente una base en específico, ingrese a charts --> databases --> values.yaml en la cual usted podrá modificar los campos enbale true = ejecutar base de datos, false = no ejecutar la base de datos.

```
elastic:  
  enabled: false #Coloque segun su preferencia  
  version: 8.6.1  
  replicas: 1 #minimo 3 datanodes  
  name: ic4302
```

Configuracion de la carga de datos

Con la finalidad de evitar el llenado de bases de datos que no estan en ejecución se establece un mecanismo similar al anterior en el cual dentro de charts --> app --> values.yaml podrá colocar en true las bbases que se desear cargar.

[!IMPORTANT]

La duracion de construccion de la imagen dataseeder puede tardar unos minutos.

```
dataseeder: # Added configuration for DataSeeder  
  enabled: true  
  name: dataseeder  
  replicas: 1  
  image: darcecampos/dataseeder # To charge with data the database  
  openSearchEnable: false  
  couchDBEnable: false  
  mariaDBEnable: false  
  elasticSearchEnable: false  
  chromaDBEnable: false  
  postgresEnable: false
```

```
mongoEnable: false
neo4jEnable: true
```

4. Instale el Helm Chart del proyecto

En su proyecto, ingrese a la carpeta de **charts** desde una terminal Bash y ejecute el siguiente comando:

```
./install.sh
```

5. Desinstalación del Helm Chart del proyecto

En caso de que usted necesite hacer la desinstalación del helm chart, ingrese a la carpeta de **charts** desde una terminal Bash y ejecute el siguiente comando:

```
./uninstall.sh
```

NOTA: Si no necesita la instalación, ignore este paso

Pruebas realizadas

► Desplegar información

OpenSearch

► Desplegar información

Backup

Crear un repositorio de Snapshots

1. Accede a OpenSearch Dashboards:

- Abre tu navegador y ve a <http://localhost:5601> (o la URL de tu OpenSearch Dashboards).
- Inicia sesión con tus credenciales (usuario: [admin](#), contraseña: configurada en [values.yaml](#)).

2. Navega a la sección de Snapshots:

- En el menú lateral izquierdo, selecciona [Stack Management](#).
- Haz clic en [Snapshot and Restore](#).

3. Registrar un repositorio de Snapshots:

- Haz clic en el botón [Register a repository](#).
- Llena el formulario con la siguiente información:
 - **Name:** Escribe un nombre para tu repositorio, por ejemplo, [opensearch-backup](#).

- **Type:** Selecciona **s3**.
- **Settings:** Copia y pega la siguiente configuración:

```
{
  "bucket": "ic-tec-dataset",
  "region": "us-east-1",
  "base_path": "CARPETA_HCDCP_BACKUP/opensearch"
}
```

- Haz clic en **Register** para guardar el repositorio.

Crear una política de Snapshots

1. Configura una política automática:

- En la sección de **Snapshot policies**.
- Haz clic en **Create policy**.
- Llena el formulario:
 - **Policy name:** Escribe un nombre, por ejemplo, **daily-backup-policy**.
 - **Repository:** Selecciona el repositorio que creaste anteriormente.
 - **Snapshot name pattern:** Usa un patrón como **backup-{now/d}** para incluir la fecha.
 - **Schedule:** Configura la frecuencia, por ejemplo, **0 0 * * *** para un respaldo diario.
- Haz clic en **Create** para guardar la política.

Crear un Snapshot manual

1. Crear un Snapshot:

- En la sección de **Snapshots**, haz clic en **Take snapshot**.
- Llena el formulario:
 - **Snapshot name:** Escribe un nombre para tu respaldo, por ejemplo, **manual-backup-2025-11-22**.
 - **Indice:** Selecciona el índice animales que se cargo previamente.
 - **Repository:** Selecciona el repositorio que creaste anteriormente.
- Haz clic en **Create** para iniciar el respaldo.

Video Tutorial: Backup

[Video Tutorial Backup](#)

Restore

Restaurar un Snapshot

1. Accede a la sección de Snapshots:

- En el menú lateral izquierdo, selecciona **Stack Management**.
- Haz clic en **Snapshot and Restore**.

2. Selecciona un Snapshot:

- En la lista de Snapshots, busca el respaldo que deseas restaurar.
- Haz clic en el botón **Restore** marcando el snapshot que deseas restaurar.

3. Configura la restauración:

- Selecciona los índices que deseas restaurar (o deja la configuración predeterminada para restaurar todo). (Animales en este caso)
- Haz clic en **Restore** para iniciar el proceso.

Verificar que el índice se generó

1. Revisar los índices restaurados:

- En el menú lateral izquierdo, selecciona **Index Management**.
- Busca el índice restaurado en la lista.
- Verifica que el estado sea **green** y que los documentos estén disponibles.

Video Tutorial: Restore

[Video Tutorial Restore](#)

-
- **Crear repositorio:**

Create repository

Repository name

s3_repo_animales

Repository type

Select a supported repository type. For additional types, install the latest repository plugins and choose Custom configuration. [Learn more](#)

Custom configuration

Install and configure custom repository types

To use a custom repository, such as Amazon S3, Azure Blob Storage or similar, install and configure the respective repository plugin on OpenSearch and then define the repository configuration below. [Learn more](#)

Custom configuration

Define a repository by custom type and settings. [View sample configurations](#)

1 {

2 "type": "s3",

3 "settings": {

4 "bucket": "ic-tec-dataset",

5 "region": "us-east-1",

6 "base_path": "CARPETA_HCDCP_BACKUP/opensearch",

7 }

8 }

Cancel

Add

- Política de Snapshots:

Policy settings

Policy name

daily_snapshot

Description - optional

Snapshot management daily policy.

Source and destination

Select or input source indexes or index patterns

animales x

x v

Select a repository for snapshots

s3_repo_animales v

Create repository

Snapshot schedule

Snapshot frequency

Daily v

Start time

 20:00

Time zone

UTC x v

- **Crear Snapshot:**

Create snapshot

Snapshot name

snapshot_animals_01

Select or input source indexes or index patterns

animales ×

⌵

Select a repository for snapshots

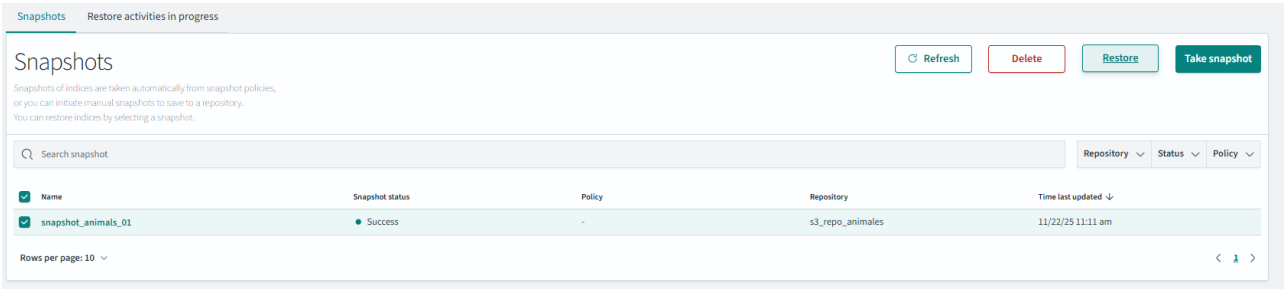
s3_repo_animales ⌵

> Advanced options

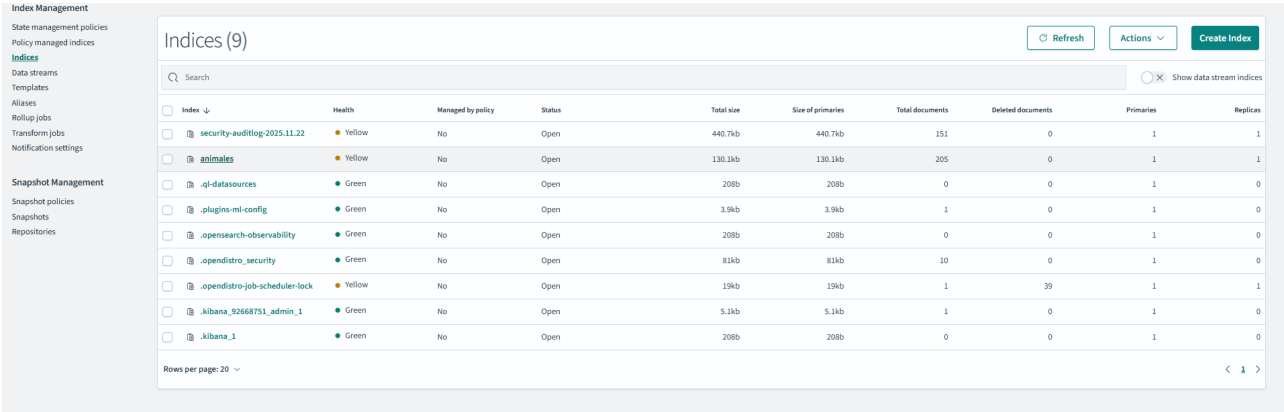
Cancel

Add

- Restaurar Snapshot:



• **Verificar índice:**



Elasticsearch

► Desplegar información

Para el desarrollo completo al realizar backups y restauraciones de indices en Elasticsearch se utiliza el servicio de Kibana al cual puede ingresar utilizando la interfaz de Lens y en la sección de Services. Una vez dentro utilizaremos su robusta configuración de Backups y restauración

[Video Tutorial Backup y Restauración](#)

Backup

Configuración del snapshot repository

Para realizar todo el procedimiento es necesario crear un Snapshot Repository, el cual es una ubicación externa donde elasticsearch puede crear y almacenar respaldos de sus indices, para poder crearlo solo debe ingresar al servicio de kibana (implementado en este proyecto) por el cual se puede interactuar con elasticsearch. En kibana ---> Stack Management ---> Snapshot y Restore ---> Repositories ---> Register repositories . Este repositorio incluye distintos parametros, para el caso de esta tarea, se deben ingresar los siguientes:

Parámetro	Valor
name	elastic
provider	AWS
client	default
bucketName	ic-tec-dataset

Parámetro	Valor
base_path	CARPETA_HCDP_BACKUP/elastic

Configuración de la política

Una vez creado el repositorio es necesario crear una política que permite ejecutar un snapshot de forma automática cada cierto tiempo y así automatizar los backups, aquí únicamente debe completar campos del repositorio ya creado.

Parámetro	Valor
name	elastic
snapshotName	elastic
repository	elastic (nombre del snapshot repository creado)

¿Cómo ejecutar el backup?

Para ejecutar el snapshot que generará el backup, puede esperar a que se ejecute según el horario que fue asignado o bien puede ejecutarlo manualmente en Kibana ---> Stack Management ---> políticas y justo en la columna de actions solicita la ejecución forzada con el botón de run.

Una vez ejecutado el policy, el backup empezará y podrá comprobarlo al ver una nueva carpeta dentro del prefijo especificado dentro del bucket, o bien puede ingresar a las Dev Tools de elastic y ejecutar:

```
GET _snapshot/elastic/_all
```

Donde en el resultado se observa cierta data que permite confirmar que el proceso fue exitoso, ya que se puede ver que parte de los índices subidos corresponde a **animals**, dataset utilizado para probar los backups

```
{
  "snapshots": [
    {
      "snapshot": "elastic-j4wjad6lsh21ww2t5mtktw",
      "repository": "elastic",
      "version_id": 8060199,
      "indices": [
        ".kibana_security_session_1",
        ".security-7",
        ".apm-agent-configuration",
        ".kibana_8.6.1_001",
        ".apm-custom-link",
        ".ds-ilm-history-5-2025.11.21-000001",
        ".geoip_databases",
        "animals",
        ".security-profile-8",
        ".kibana-event-log-8.6.1-000001",
        ".ds-.logs-deprecation.elasticsearch-default-2025.11.21-000001",
        ".kibana_task_manager_8.6.1_001"
      ]
    }
  ]
}
```

```
],  
"metadata": {  
  "policy": "elastic"  
},  
"state": "SUCCESS",
```

Restauración

Antes de realizar el proceso para hacer la restauración de índices es importante mencionar que Elasticsearch **NO** permite restaurar un índice si este ya existe o está abierto, es por esta razón que para probar la restauración es necesario eliminar el índice `animals` antes de ejecutar el restore, utilizando este comando en la consola de Dev-Tools se puede realizar la eliminación:

```
DELETE animals
```

Una vez realizado esto puede dirigirse a Stack Management ---> Snapshot y Restore ---> Snapshots y justo en la columna de Actions seleccionar la opción `Restore`. También es importante mencionar que al eliminar únicamente el índice `animals`, podremos hacer restore de únicamente ese índice, ya que los demás son los creados por el sistema y decidimos mejor no restaurarlos para evitar permisos u otros problemas que puedan surgir, por esta razón, al seleccionar esta opción de restore, debe ajustar la configuración para que solo se haga restore del índice creado por el usuario, en este caso `animals` de esta manera:

 Captura de pantalla 2025-11-21 200911

Una vez realizado esto puede ejecutar el snapshot completo y verificar que el índice vuelve a ser creado

Crear snapshot repository

 Captura de pantalla 2025-11-24 104159  Captura de pantalla 2025-11-24 104247

Crear política

 Captura de pantalla 2025-11-24 105120

Ejecutar backup manual

 Captura de pantalla 2025-11-24 105154

Realizar restore

 Captura de pantalla 2025-11-24 110405

Mongo

► Desplegar información

Este proyecto implementa un sistema completo de backup y restauración para MongoDB usando AWS S3 como almacenamiento externo y Scripts Bash. La solución permite realizar respaldos automáticos y restauraciones de la base de datos animalsdb, utilizada como dataset de prueba.

Backup

El proceso de respaldo se realiza automáticamente mediante un CronJob de Kubernetes que se ejecuta automáticamente cada 12 horas. El respaldo genera un archivo comprimido .gz con formato:

```
YYYYmmDDHHMM.gz
```

Dentro del `values.yaml` debe estar habilitado el modulo de la siguiente manera:

- `connectionString`: dirección interna del servicio de MongoDB.
- `bucketName`: nombre del bucket de AWS S3 donde se almacenarán los respaldos
- `path`: Ruta dentro del bucket donde se guardarán/leerán los respaldos.
- `type`:
 - `backup`: se genera el cronjob de respaldo
 - `restore`: se genera el ob de restauración

```
mongo:
  enabled: true
  config:
    namespace: default
    connectionString: databases-mongodb.default.svc.cluster.local:27017
    bucketName: ic-tec-dataset
    path: CARPETA_HCDCP_BACKUP/mongodb
    maxBackups: 3
    secret: databases-mongodb
    name: "202511230002" # Nombre de la copia de seguridad a
  descargar
    schedule: "0 */12 * * *"
    diskSize: 2
    storageClass: hostpath
    provider: aws
    type: backup
```

Restore

El proceso de restauración se ejecuta mediante un Job que descarga el archivo de respaldo desde S3 y lo importa en MongoDB. Este sistema permite restaurar solo la base animalsdb, evitando sobrescribir la base admin o los usuarios del clúster. En `values.yaml` se debe utilizar el parametro `name` que define el archivo .gz a restaurar que se encuentra dentro del s3 bucket. Además, el parametro type siempre deberá ser **restore**.

```
mongo:
  enabled: true
  config:
    name: "202511222147"
    type: restore
```

Cómo saber si mi backup o restore salieron bien?

Para confirmar ambos procesos puede revisar dentro del bucket y verificar que hay nuevos archivos que corresponden a la fecha y hora en el que fue ejecutado el proceso. Para revisar el bucket puede utilizar el siguiente comando en PowerShell:

```
aws s3 ls s3://ic-tec-dataset/CARPETA_HCDCP_BACKUP/mongodb/
```

Nota: Antes debió ingresar sus credenciales para poder entrar al s3 bucket.

Dentro de esta carpeta podrá confirmar el archivo al cual se le hizo backup, así como elegir el nombre del archivo al cual le quiere hacer una restauración de datos.

MariaDB

► Desplegar información

Este proyecto implementa un sistema completo de backup y restauración para MariaDB usando AWS S3 como almacenamiento externo y Scripts Bash. La solución permite realizar respaldos automáticos y restauraciones de todas las bases de datos, manteniendo las n copias de seguridad más recientes de forma configurable (por defecto 3)

Backup

El proceso de respaldo se realiza automáticamente mediante un CronJob de Kubernetes que se ejecuta cada 12 horas. El respaldo genera un archivo SQL plano con formato:

```
YYYYmmDDHHMM.gz
```

Dentro del `values.yaml` debe estar habilitado el modulo de la siguiente manera:

- `connectionString`: Dirección interna del servicio de MariaDB (ej. `databases-mariadb.default.svc.cluster.local:3306`).
- `bucketName`: Nombre del bucket de AWS S3 donde se almacenarán los respaldos
- `path`: Ruta dentro del bucket donde se guardarán/leerán los respaldos.
- `maxBackups`: Número máximo de backups a mantener.
- `type`:
 - `backup`: Se genera el cronjob de respaldo
 - `restore`: Se genera el Job de restauración (requiere especificar name con el archivo a restaurar)

- name: Nombre del backup a restaurar (solo para type restore: 202511240000.sql).

```
mariadb:
  enabled: true
  config:
    namespace: default
    connectionString: databases-mariadb.default.svc.cluster.local:3306
    bucketName: ic-tec-dataset
    path: CARPETA_HCDP_BACKUP/mariadb
    maxBackups: 3
    secret: databases-mariadb
    name: mariadb
    schedule: "0 */12 * * *"
    diskSize: 2Gi
    storageClass: hostpath
    provider: aws
    type: backup # Cambiar a 'restore' para restaurar
    # name: 202511240000.sql # Descomentar para restore
    serviceAccount: default
```

Restore

Para restaurar, cambia type a restore y especifica el name del archivo en S3. El Job sobrescribirá la base de datos actual.

Notas importantes

- Los backups se eliminan automáticamente si exceden `maxBackups` para mantener solo las copias más recientes.
- Los scripts Bash deben guardarse con finales de línea LF (Unix), no CRLF (Windows). En Visual Studio Code, verificar la esquina inferior derecha (debe decir "LF"). Si dice "CRLF", darle clic y seleccionar "LF" para evitar errores en los pods de Kubernetes.

PostgreSQL

► Desplegar información

Este proyecto implementa un sistema completo de backup y restauración para PostgreSQL usando AWS S3 como almacenamiento externo y Scripts Bash. La solución permite realizar respaldos automáticos y restauraciones de todas las bases de datos, manteniendo las n copias de seguridad más recientes de forma configurable (por defecto 3)

Backup

El proceso de respaldo se realiza automáticamente mediante un CronJob de Kubernetes que se ejecuta cada 12 horas. El respaldo genera un archivo SQL plano con formato:

```
YYYYmmDDHHMM.gz
```

Dentro del `values.yaml` debe estar habilitado el modulo de la siguiente manera:

- `connectionString`: Dirección interna del servicio de PostgreSQL (ej. `databases-postgresql.default.svc.cluster.local:5432`).
- `bucketName`: Nombre del bucket de AWS S3 donde se almacenarán los respaldos
- `path`: Ruta dentro del bucket donde se guardarán/leerán los respaldos.
- `maxBackups`: Número máximo de backups a mantener.
- `type`:
 - `backup`: Se genera el cronjob de respaldo
 - `restore`: Se genera el Job de restauración (requiere especificar name con el archivo a restaurar)
 - `name`: Nombre del backup a restaurar (solo para type restore: `202511240000.sql`).

```
postgresql:
  enabled: true
  config:
    namespace: default
    connectionString: databases-postgresql.default.svc.cluster.local:5432
    bucketName: ic-tec-dataset
    path: CARPETA_HCDCP_BACKUP/postgresql
    maxBackups: 3
    secret: databases-postgresql
    name: postgres
    schedule: "0 */12 * * *"
    diskSize: 2Gi
    storageClass: hostpath
    provider: aws
    type: backup # backup o restore
    # name: 202511232330.sql # nombre del backup a restaurar
    serviceAccount: default
```

Restore

Para restaurar, cambia type a restore y especifica el name del archivo en S3. El Job sobrescribirá la base de datos actual.

Notas importantes

- Los backups se eliminan automáticamente si exceden `maxBackups` para mantener solo las copias más recientes.
- Los scripts Bash deben guardarse con finales de línea LF (Unix), no CRLF (Windows). En Visual Studio Code, verificar la esquina inferior derecha (debe decir "LF"). Si dice "CRLF", darle clic y seleccionar "LF" para evitar errores en los pods de Kubernetes.

CouchDB

► Desplegar información

Este proyecto implementa un sistema completo de backup y restauración para CouchDB usando AWS S3 como almacenamiento externo y Scripts Bash. La solución permite realizar respaldos automáticos y restauraciones de la base de datos animalsdb, utilizada como dataset de prueba.

Backup

El proceso de respaldo se realiza automáticamente mediante un CronJob de Kubernetes que se ejecuta automáticamente cada 12 horas. El respaldo genera un archivo comprimido .gz con formato:

```
YYYYmmDDHHMM.gz
```

Dentro del `values.yaml` debe estar habilitado el modulo de la siguiente manera:

- `conectionString`: dirección interna del servicio de CouchDB.
- `bucketName`: nombre del bucket de AWS S3 donde se almacenarán los respaldos
- `path`: Ruta dentro del bucket donde se guardarán/leerán los respaldos.
- `type`:
 - `backup`: se genera el cronjob de respaldo
 - `restore`: se genera el job de restauración

```
couchdb:
  enabled: true
  config:
    namespace: default
    connectionString: databases-couchdb.default.svc.cluster.local:5984
    bucketName: ic-tec-dataset
    path: CARPETA_HCDCP_BACKUP/couchdb
    maxBackups: 3
    secret: databases-couchdb
    name: "202511280227" # Nombre de la copia de seguridad a
  descargar
    schedule: "0 */12 * * *"
    diskSize: 2
    storageClass: hostpath
    provider: aws
    type: backup
    serviceAccount: default
```

Restore

El proceso de restauración se ejecuta mediante un Job que descarga el archivo de respaldo desde S3 y lo sube a CouchDB. Para esto se elimina la base de datos, se vuelve a crear la base de datos vacía y se suben todos los

documentos guardados en el backup. Este sistema permite restaurar solo la base animalsdb. En `values.yaml` se debe utilizar el parametro `name` que define el archivo .gz a restaurar que se encuentra dentro del s3 bucket. Si no se asigna un backup existente, da un mensaje de error.

```
couchdb:
  enabled: true
  config:
    namespace: default
    connectionString: databases-couchdb.default.svc.cluster.local:5984
    bucketName: ic-tec-dataset
    path: CARPETA_HCDCP_BACKUP/couchdb
    maxBackups: 3
    secret: databases-couchdb
    name: "202511280227" # Nombre de la copia de seguridad a
descargar
  schedule: "0 */12 * * *"
  diskSize: 2
  storageClass: hostpath
  provider: aws
  type: restore #Cambiar a restore para restaurar
  serviceAccount: default
```

Notas importantes

- Los scripts Bash deben guardarse con finales de línea LF (Unix), no CRLF (Windows). En Visual Studio Code, verificar la esquina inferior derecha (debe decir "LF"). Si dice "CRLF", darle clic y seleccionar "LF" para evitar errores en los pods de Kubernetes.

Neo4j

► Desplegar información

En el presente proyecto se implementa un sistema completo de backup y restauración para Neo4j usando AWS S3 como almacenamiento externo y Scripts Bash. La solución permite realizar respaldos automáticos y restauraciones de la base de datos animalsdb, utilizada en este caso como dataset de prueba.

Backup

El proceso de respaldo se realiza automáticamente mediante un CronJob de Kubernetes que se ejecuta automáticamente cada 12 horas (en caso de querer cambiarlo, se puede editar en `helm/backups/values.yaml`). El respaldo genera un archivo comprimido .gz con formato:

```
YYYYmmDDHHMM.gz
```

Dentro del `values.yaml` debe estar habilitado el modulo de la siguiente manera:

- `conectionString`: dirección interna del servicio de Neo4j.
- `bucketName`: nombre del bucket de AWS S3 a utilizar para almacenar los respaldos.

- **path**: Ruta dentro del bucket donde se guardarán/leerán los respaldos.
- **type**:
 - backup: se genera el cronjob de respaldo de la base.
 - restore: se genera el job de restauración de la base.

```
neo4j:
  enabled: true
  config:
    namespace: default
    connectionString: databases-neo4j.default.svc.cluster.local:7687
    bucketName: ic-tec-dataset
    path: CARPETA_HCDCP_BACKUP/neo4j
    maxBackups: 3
    secret: databases-neo4j
    name: neo4j
    schedule: "0 */12 * * *"
    diskSize: 2Gi
    storageClass: hostpath
    provider: aws
    type: backup #se puede cambiar a restore
    serviceAccount: default
```

Restore

El proceso de restauración se ejecuta mediante un Job que descarga el archivo de respaldo desde S3 y lo sube a Neo, permitiendo restaurar la base de datos animalsdb. En `values.yaml` se debe utilizar el parametro `name` que define el archivo .gz a restaurar que se encuentra dentro del s3 bucket. Si no se asigna un backup existente, y se deja en blanco el espacio, se restaurará el último backup. Se debe cambiar el type a restore.

```
neo4j:
  enabled: true
  config:
    namespace: default
    connectionString: databases-neo4j.default.svc.cluster.local:7687
    bucketName: ic-tec-dataset
    path: CARPETA_HCDCP_BACKUP/neo4j
    maxBackups: 3
    secret: databases-neo4j
    name: "202511280227"
    schedule: "0 */12 * * *"
    diskSize: 2Gi
    storageClass: hostpath
    provider: aws
    type: backup #se puede cambiar a restore
    serviceAccount: default
```

Nombre

- Uno de los parámetros que pide Neo4j es el nombre a utilizar, se puede elegir libremente mientras no exista:

```
neo4j:
  name: "databases-neo4j"
```

Configuración de componentes

► Desplegar información

Llenado de las bases de datos

► Desplegar información

Dataset de Prueba: Información de Animales

Este apartado describe el dataset utilizado como prueba en el proyecto.
El dataset proporciona información general sobre diversas especies animales, abarcando datos físicos, ecológicos, taxonómicos y de comportamiento.

Fuente original: [Animal Information Dataset - Kaggle](#)

Descripción General

El dataset reúne un conjunto de características asociadas a distintos animales, como su altura, peso, dieta, hábitat, depredadores, estado de conservación, entre otros.
Este recurso fue empleado como dataset de prueba para validar funcionalidades en el proyecto, ya que contiene datos variados que permiten realizar consultas, análisis y visualizaciones desde distintos enfoques.

Estructura del Dataset

El dataset está compuesto por diversas columnas (atributos) que describen a cada animal. A continuación se detalla el glosario columna por columna:

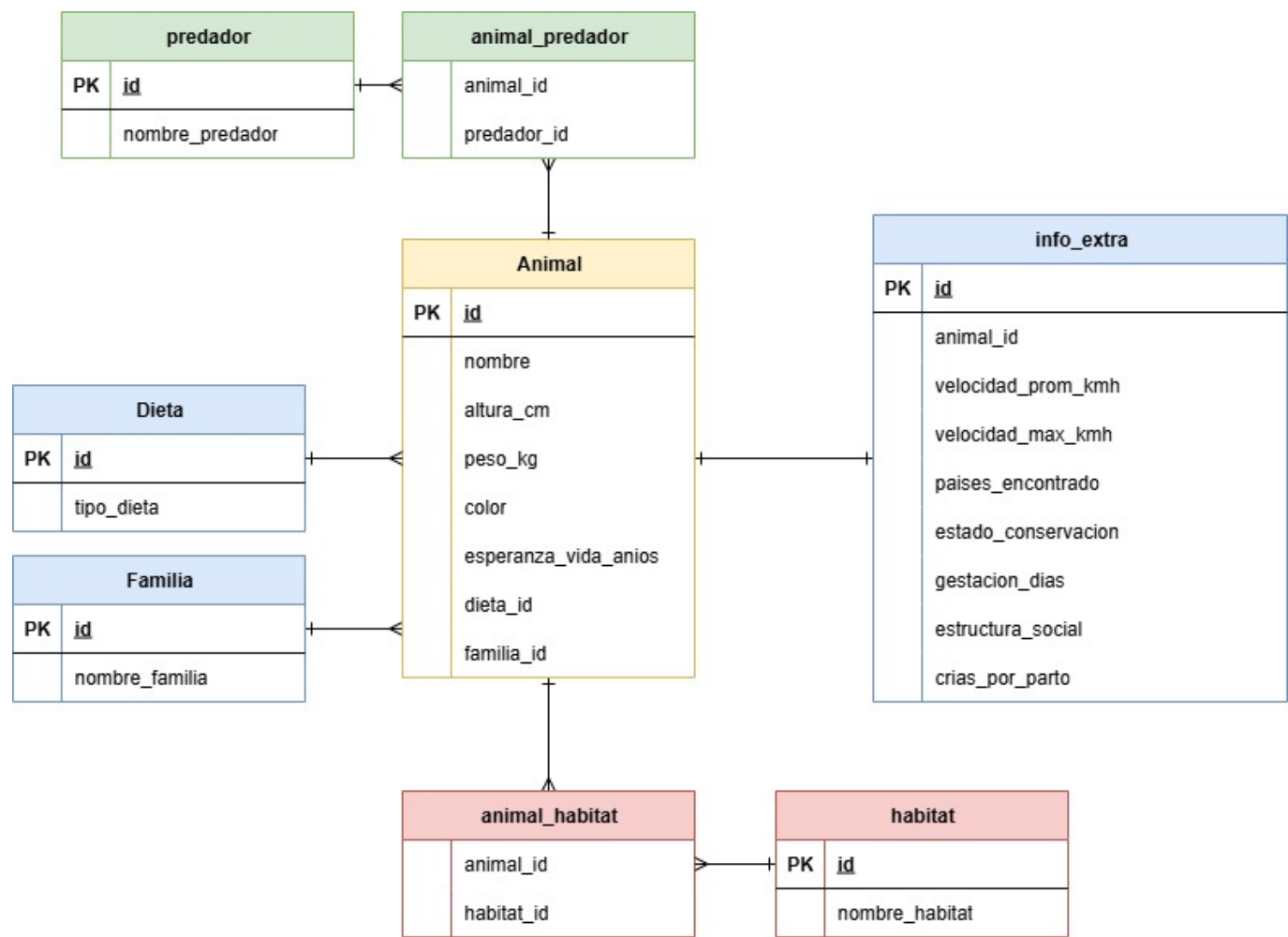
Columna	Descripción
Animal	Nombre del animal.
Height (cm)	Rango de altura en centímetros del animal.
Weight (kg)	Rango de peso en kilogramos del animal.
Color	Colores comunes asociados a la apariencia del animal.

Columna	Descripción
Lifespan (years)	Promedio de años de vida del animal.
Diet	Tipo de dieta que sigue principalmente (Ejemplo: carnívoro, herbívoro).
Habitat	Hábitat o entorno natural donde suele encontrarse el animal.
Predators	Depredadores o enemigos naturales del animal.
Average Speed (km/h)	Velocidad promedio en kilómetros por hora.
Countries Found	Países o regiones donde el animal es comúnmente encontrado.
Conservation Status	Estado de conservación según organismos especializados (Ejemplo: En peligro).
Family	Familia taxonómica a la que pertenece el animal.
Gestation Period (days)	Rango en días del período de gestación o embarazo.
Top Speed (km/h)	Velocidad máxima alcanzable en kilómetros por hora.
Social Structure	Estructura social o comportamiento (Ejemplo: solitario, en grupo).
Offspring per Birth	Número típico de crías por evento reproductivo.

Esquema Relacional del Dataset de Animales

Este apartado describe la estructura de base de datos SQL diseñada para almacenar el dataset de prueba sobre animales.

La implementación sigue un modelo relacional normalizado, con separación de entidades principales y relaciones uno a muchos y muchos a muchos.



Mapeo de las bases documentales

```
{
  "animals": {
    "mappings": {
      "properties": {
        "average_speed_kmh": {
          "type": "keyword"
        },
        "color": {
          "type": "keyword"
        },
        "conservation_status": {
          "type": "keyword"
        },
        "countries_found": {
          "type": "text"
        },
        "diet": {
          "type": "keyword"
        },
        "family": {
          "type": "keyword"
        }
      }
    }
  }
}
```

```
    "gestation_period_days": {
      "type": "keyword"
    },
    "habitat": {
      "type": "text"
    },
    "height_cm": {
      "type": "keyword"
    },
    "lifespan_years": {
      "type": "keyword"
    },
    "name": {
      "type": "keyword"
    },
    "offspring_per_birth": {
      "type": "keyword"
    },
    "predators": {
      "type": "text"
    },
    "social_structure": {
      "type": "text"
    },
    "top_speed_kmh": {
      "type": "keyword"
    },
    "weight_kg": {
      "type": "keyword"
    }
  }
}
```

Opensearch

► Desplegar información

¿Qué es OpenSearch?

OpenSearch es una suite de software de código abierto diseñada para búsquedas y análisis de datos en tiempo real. Es un fork de Elasticsearch 7.10, desarrollado por Amazon Web Services (AWS) y la comunidad, con el objetivo de proporcionar una alternativa completamente abierta y gratuita. OpenSearch permite indexar, buscar y analizar grandes volúmenes de datos, siendo ideal para casos de uso como monitoreo de logs, análisis de datos, búsqueda en aplicaciones y más.

Componentes principales de OpenSearch:

1. **OpenSearch:** El motor principal que permite almacenar, buscar y analizar datos.
2. **OpenSearch Dashboards:** Una interfaz gráfica basada en web que facilita la visualización y gestión de los datos almacenados en OpenSearch. Es similar a Kibana, pero completamente de código abierto.

Configuración de OpenSearch en este proyecto

En este proyecto, OpenSearch se utiliza para almacenar y analizar datos relacionados con animales. La configuración incluye tanto el motor de OpenSearch como OpenSearch Dashboards para la visualización de datos.

Configuración básica en `values.yaml`

El archivo `values.yaml` contiene los parámetros necesarios para desplegar OpenSearch y OpenSearch Dashboards en un clúster de Kubernetes. A continuación, se muestra un ejemplo de configuración:

```
opensearch:
  enabled: true
  fullnameOverride: "mi-opensearch"
  clusterName: "opensearch-cluster"
  replicas: 1
  extraEnvs:
    - name: OPENSEARCH_INITIAL_ADMIN_PASSWORD
      value: "BaseDatos12345!"
opensearchDashboards:
  enabled: true
  fullnameOverride: "mi-opensearch-dashboards"
  opensearchHosts:
    - "http://opensearch-cluster-master.default.svc.cluster.local:9200"
```

OpenSearch Dashboards

OpenSearch Dashboards es una herramienta visual que permite interactuar con los datos almacenados en OpenSearch. Con Dashboards, puedes:

- Crear visualizaciones y gráficos interactivos.
- Configurar paneles personalizados para monitorear datos en tiempo real.
- Gestionar índices y configuraciones de OpenSearch.

Para acceder a OpenSearch Dashboards, puedes utilizar herramientas como Lens para una experiencia más visual e interactiva. Lens es una extensión que permite explorar y analizar datos de manera intuitiva. Sigue estos pasos para configurarlo:

1. **Instalar Lens:** Descarga e instala Lens desde su sitio oficial: <https://k8slens.dev/>.
2. **Conectar Lens a tu clúster de Kubernetes:**
 - Abre Lens y agrega tu clúster de Kubernetes.
 - Asegúrate de que el clúster esté configurado correctamente y que tengas acceso a los recursos necesarios.
3. **Acceder a OpenSearch Dashboards:**
 - En Lens, navega a la sección de servicios.
 - Busca el servicio `mi-opensearch-dashboards`.

- Haz clic en el botón forward para abrir el servicio en tu navegador.

4. Iniciar sesión en OpenSearch Dashboards:

- Utiliza las credenciales configuradas en `values.yaml` (por defecto, usuario: `admin`, contraseña: la definida en `OPENSEARCH_INITIAL_ADMIN_PASSWORD`).

Advertencia

- **OpenSearch y Elasticsearch no pueden ejecutarse simultáneamente en el mismo clúster.**
Asegúrate de que en el archivo `values.yaml`, la variable `enabled` de uno de ellos esté configurada en `false` si el otro está en `true`. Esto evitará conflictos entre los servicios.

Elasticsearch

► Desplegar información

Elasticsearch es un motor distribuido de búsqueda, análisis y almacenamiento de datos en tiempo real. Es ampliamente utilizado en la industria gracias a su capacidad para indexar y consultar grandes volúmenes de información de manera rápida.

Fue creado originalmente por Elastic NV y se ha convertido en uno de los sistemas de búsqueda y análisis más populares del mundo debido a su flexibilidad, escalabilidad y ecosistema de herramientas complementarias como Kibana.

Configuración de Elasticsearch en este proyecto

En este proyecto, Elasticsearch se utilizó para almacenar, indexar y consultar información relacionada con animales, generada mediante el dataseeder. La instalación se realizó utilizando Helm Charts.

El archivo `values.yaml` controla la configuración del cluster y Kibana, permitiendo habilitar o deshabilitar estos servicios según se necesite.

```
elastic:
  enabled: true
  version: 8.6.1
  replicas: 1
  name: ic4302
  fullnameOverride: mi-elasticsearch
kibana:
  enabled: true
  version: 8.6.1
  replicas: 1
  name: ic4302
```

Kibana en este proyecto

Kibana fue utilizado para:

- Consultar los índices generados por el dataseeder

- Crear el repositorio S3 para snapshots
- Configurar la política automática de backup
- Ejecutar snapshots manuales
- Realizar el proceso de restore
- Visualizar información del sistema, mappings, documentos, etc.

Para acceder a Kibana, se utilizó port-forwarding en la interfaz provista por Lens.

Mongo

► Desplegar información

En este proyecto, MongoDB se utilizó para almacenar los datos generados por el dataseeder, específicamente la base de datos animalsdb utilizada para los procesos de análisis y pruebas. Para administrar backups y restauraciones de esta base, se creó un Helm Chart que se encarga de:

- Generar respaldos automáticos en formato .gz
- Guardarlos en un bucket de AWS S3
- Ejecutar restauraciones controladas bajo demanda

El archivo values.yaml dentro de la carpeta de bases de datos controla la configuración del cluster, permitiendo habilitar o deshabilitar este servicio según se necesite.

```
mongodb:
  image:
    repository: bitnamilegacy/mongodb
  global:
    security:
      allowInsecureImages: true
  enabled: true
  livenessProbe:
    enabled: true
  readinessProbe:
    enabled: true
  extraEnvVars:
    - name: EXPERIMENTAL_DOCKER_DESKTOP_FORCE_QEMU
      value: "1"
```

MariaDB

► Desplegar información

En este proyecto, MariaDB se utilizó para almacenar los datos generados por el dataseeder, específicamente bases de datos relacionales utilizadas para los procesos de análisis y pruebas. Para administrar backups y restauraciones de estas bases, se creó un Helm Chart que se encarga de:

- Generar respaldos automáticos en formato .sql
- Guardarlos en un bucket de AWS S3
- Ejecutar restauraciones controladas bajo demanda

- Mantener las n copias de seguridad más recientes de forma configurable

El archivo values.yaml dentro de la carpeta de bases de datos controla la configuración del cluster, permitiendo habilitar o deshabilitar este servicio según se necesite.

```
mariadb:
  replicas: 1
  enabled: true
  image:
    registry: docker.io
    repository: bitnamilegacy/mariadb
    tag: 11.2.3-debian-11-r1
```

PostgreSQL

► Desplegar información

En este proyecto, PostgreSQL se utilizó para almacenar los datos generados por el dataseeder, específicamente bases de datos relacionales utilizadas para los procesos de análisis y pruebas. Para administrar backups y restauraciones de estas bases, se creó un Helm Chart que se encarga de:

- Generar respaldos automáticos en formato .sql
- Guardarlos en un bucket de AWS S3
- Ejecutar restauraciones controladas bajo demanda
- Mantener las n copias de seguridad más recientes de forma configurable

El archivo values.yaml dentro de la carpeta de bases de datos controla la configuración del cluster, permitiendo habilitar o deshabilitar este servicio según se necesite.

```
postgresql:
  replicas: 1
  enabled: true
  image:
    registry: docker.io
    repository: bitnamilegacy/postgresql
    tag: 15.5.0-debian-11-r1
```

Neo4j

► Desplegar información

¿Qué es Neo4j?

Neo4j es una base de datos orientada a grafos, que está diseñada para modelar y consultar relaciones complejas entre entidades. A diferencia de las bases relacionales o las NoSQL más tradicionales, Neo4j almacena la información como nodos, relaciones y propiedades, lo que la vuelve extremadamente eficiente en

las consultas sobre redes, jerarquías o cualquier tipo de estructura conectada. Neo4j utiliza el lenguaje Cypher, diseñado para consultar grafos de forma declarativa e intuitiva.

Es el motor de grafos más utilizado en la industria para ciertos casos de uso como por ejemplo:

- Redes sociales
- Sistemas de recomendación
- Grafos de conocimiento
- Análisis de rutas
- Detección de fraude
- Modelos con relaciones naturales y dinámicas

Configuración

En este proyecto, Neo4j se utilizó como base de datos de grafos para almacenar relaciones complejas de los datos generados por el dataseeder. Al igual que con los otros motores, se implementó un sistema completo de backups y restauraciones automatizadas

Todo esto es configurable mediante el archivo values.yaml, el cual controla tanto la configuración de Neo y los parámetros del sistema de backup, permitiendo ajustar este servicio según las necesidades, siendo todo parametrizable como ya se vio antes que también se puede editar los backups:

```
neo4j:
  enabled: true

neo4j:
  name: "databases-neo4j"
  password: "Neo4j123!"
  edition: "community"
  acceptLicenseAgreement: "yes"

volumes:
  data:
    mode: "defaultStorageClass"
    defaultStorageClass:
      accessModes:
        - ReadWriteOnce
    requests:
      storage: 5Gi

services:
  neo4j:
    enabled: true
    spec:
      type: ClusterIP

auth:
  username: "neo4j"
  password: "Neo4j123!"
image:
  registry: docker.io
```

```
repository: bitnamilegacy/neo4j
tag: 5.18.0
```

CouchDB

► Desplegar información

¿Qué es CouchDB?

CouchDB es una excelente base de datos de un solo nodo que funciona como cualquier otra base de datos, respaldada por un servidor de aplicaciones de su elección. CouchDB utiliza el protocolo HTTP y el formato de datos JSON, ampliamente utilizados, y es compatible con cualquier software que los admita.

Configuración

En este proyecto, CouchDB se utilizó como base de datos NoSQL orientada a documentos para almacenar la información generada por el dataseeder, específicamente para manejar datos flexibles basados en JSON que no requieren un esquema fijo. Para administrar backups y restauraciones de estas bases, se creó un Helm Chart que se encarga de:

- Generar respaldos automáticos en formato .json
- Guardarlos en un bucket de AWS S3
- Ejecutar restauraciones controladas bajo demanda

El archivo values.yaml controla tanto la configuración del cluster CouchDB como los parámetros del sistema de backup, permitiendo ajustar este servicio según las necesidades del entorno.

```
couchdb:
  enabled: true
  clusterSize: 1
  allowAdminParty: false
  createAdminSecret: true
  adminUsername: admin
  adminPassword: "Admin123!"
  adminHash: ""
  cookieAuthSecret: ""
  image:
    repository: couchdb
    tag: 3.5.0
    pullPolicy: IfNotPresent
  service:
    enabled: true
    type: ClusterIP
    externalPort: 5984
    targetPort: 5984
  couchdbConfig:
    couchdb:
      uuid: "2f0e4cb63d984ea5b77b4b9a6fd43f19"
    chttpd:
```

```
bind_address: any
require_valid_user: false
```

Conclusiones

► Desplegar información

1. El proceso de realizar backups y restores con Elasticsearch evidencia que el proceso que tienen de snapshots y restore es bastante robusto y bastante facil de utilizar, es decir no deja de ser seguro para los momentos en los que la disponibilidad de los datos juega un papel critico.
2. El uso de interfaces como el dashboard de OpenSearch facilita enormemente la generación de backups y restores, ya que proporciona herramientas visuales intuitivas que permiten gestionar snapshots, configurar repositorios y restaurar índices de manera eficiente, sin necesidad de depender exclusivamente de comandos en la terminal.
3. OpenSearch es una base de datos similar a Elasticsearch, ya que ambas están diseñadas para búsquedas y análisis de datos en tiempo real. Sin embargo, OpenSearch es un proyecto de código abierto completamente independiente, desarrollado como un fork de Elasticsearch 7.10, y no incluye las características propietarias de Elastic. Además, OpenSearch pone un mayor énfasis en la transparencia y la comunidad, mientras que Elasticsearch incluye funcionalidades avanzadas bajo licencias comerciales.
4. Guardar archivos con formato YYYYmmDDHHMM.gz es una excelente opcion para facilitar la organización y trazabilidad de los respaldos.
5. El mecanismo de mantener las n copias de seguridad más recientes en MariaDB y PostgreSQL es altamente efectivo para optimizar el almacenamiento en S3, ya que elimina automáticamente los backups antiguos, evitando costos innecesarios y facilitando la gestión de versiones sin intervención manual.
6. La implementación de backups en formato SQL plano para bases relacionales como MariaDB y PostgreSQL permite una restauración precisa y compatible con cualquiera.
7. El uso de CouchDB y su formato NoSQL basado en json facilitó en gran medida el proceso de backup y restore, haciendo que sea una base de datos muy conveniente y segura.
8. El uso de Shell Scripts facilitó el proceso de backups y restores, ya que permitía manejar los archivos, las operaciones crud con curl y la edición de texto de manera muy sencilla.
9. La integración con S3 unifica la gestión de respaldos entre motores, permitiendo un enfoque uniforme para todas las bases de datos que puedan llegar a estar presentes dentro de un proyecto.
10. Neo4j es una base de datos muy eficiente para manejar datos con estructuras complejas, siendo una gran opción para realizar respaldos de datasets con muchas estructuras.

Recomendaciones

► Desplegar información

1. Ya que elasticsearch funciona con índices que cambian de manera dinámica es bastante ventajoso apoyarse de snapshots automáticos mediante politicas que se adecuen a las necesidades del negocio y así evitar fallo o algun borrado accidental.
2. Es recomendable almacenar las credenciales sensibles, como el usuario y la contraseña de OpenSearch (OPENSEARCH_USER y OPENSEARCH_PASS), en un sistema de gestión de secretos o en un archivo de

configuración seguro, en lugar de incluirlas directamente en el archivo dataseeder.yaml.

3. Antes de realizar un backup en OpenSearch, verificar que el índice al que se le está haciendo el snapshot no sea un índice protegido o del sistema (como .security-* o .kibana-*), ya que estos índices pueden requerir permisos adicionales o configuraciones específicas. Si intentas realizar un backup de estos índices sin los permisos adecuados, el proceso puede fallar. Para evitar errores, asegúrate de incluir únicamente los índices necesarios en la configuración del snapshot o utiliza un filtro para excluir los índices protegidos.
4. Verificar que antes de realizar la ejecución de los restores en este proyecto, se encuentre habilitada la opción de `type: restore` para que así se ejecute coreectamente este proceso asi como verificar la restauración especifica que se quiere realizar por medio de la variable `name`.
5. Configurar el parámetro maxBackups en el values.yaml de backups según las necesidades, considerando factores como frecuencia de cambios en los datos.
6. Antes de ejecutar backups o restores en MariaDB y PostgreSQL, verificar que los scripts Bash estén guardados con finales de línea LF (Unix) en el editor (ej. VS Code), no CRLF (Windows), para evitar errores de ejecución en los pods de Kubernetes.
7. Mantener la consistencia en el nombre de los archivos de backups, y particularmente usar el formato YYYYmmDDHHMM es muy recomendable ya que permite ordenar los backups más fácilmente y hacer restauraciones automáticas.
8. Se recomienda no escribir ni iniciar procesos concurrentes el la base de datos mientras se hace un restore, ya que esto puede generar fallos e inconsistencias.
9. Se recomienda configurar el horario del cronjob de restore en el values.yaml de backups, bases con datos masivos y constantes cambios suelen necesitar backups constantes.
10. A la hora de configurar un ambiente con neo4j, se recomienda mantener deshabilitado Neo4j Enterprise excepto cuando sea estrictamente necesario, ya que activa características que requieren licencia.

Referencias

► Desplegar información

<https://docs.opensearch.org/latest/install-and-configure/install-opensearch/helm/>

<https://docs.opensearch.org/latest/install-and-configure/install-dashboards/helm/>

<https://docs.opensearch.org/latest/api-reference/snapshots/restore-snapshot/>

<https://docs.opensearch.org/latest/api-reference/snapshots/create-snapshot/>

<https://docs.opensearch.org/latest/install-and-configure/plugins/>

<https://www.elastic.co/search-labs/blog/how-do-incremental-snapshots-work>

<https://www.mongodb.com/docs/database-tools/mongorestore/>

<https://couchdb.apache.org/>

<https://docs.couchdb.org/en/stable/install/kubernetes.html> <https://github.com/apache/couchdb-helm>

<https://docs.couchdb.org/en/stable/api/>

<https://docs.couchdb.org/en/stable/api/database/common.html#put--db>

<https://docs.couchdb.org/en/stable/api/database/common.html#post--db>

https://janl.github.io/couchdb-docs/couchdb-manual-1.1/couchdb-manual.html-section/couchdb-api-db_db-bulk-docs_post.html

<https://moldstud.com/articles/p-creating-and-managing-couchdb-databases-with-python-an-easy-guide>

<https://docs.couchdb.org/en/stable/api/database/bulk-api.html#db-all-docs>

<https://docs.couchdb.org/en/stable/maintenance/backups.html>

<https://github.com/neo4j/helm-charts/tree/dev/neo4j>

<https://neo4j.com/docs/operations-manual/current/kubernetes/quickstart-cluster/create-value-file/>

<https://neo4j.com/>

<https://github.com/neo4j/neo4j>

<https://www.gnu.org/software/gzip/manual/gzip.html>

<https://jqlang.org/manual/#array-construction>

Tabla de Estado

► Desplegar información

Componente / Requerimiento	% Evaluación	Estado
MariaDB – Backup/Restore	10%	Implementado
PostgreSQL – Backup/Restore	10%	Implementado
MongoDB – Backup/Restore	10%	Implementado
Neo4J – Backup/Restore	10%	Implementado
CouchDB – Backup/Restore	10%	Implementado
Elasticsearch – Backup/Restore	10%	Implementado
OpenSearch – Backup/Restore	10%	Implementado